

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	G. Naveen Kumar Reddy	Reg.No.:	192525288
Department:	AIML	Date:	

ANNEXURE A
Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I **G. Naveen Kumar Reddy (192525288)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **G. Naveen Kumar Reddy**

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ANSWERS

Constraint-Based Problem Solving – CO5: I/O Communication & Interrupt Handling

Question 1: Real-Time Embedded I/O Communication Mechanism

Problem Understanding

The system must service interrupts from multiple sensors arriving every 1 ms, while keeping CPU utilization under 15%, using a vectored interrupt scheme, and choosing the address-mapping technique (memory-mapped or I/O-mapped) that gives the lowest latency.

Constraint Analysis

- CPU utilization < 15% → rules out busy-wait polling; the CPU must remain mostly idle/available for other tasks, so an interrupt-driven approach is mandatory.
- Interrupts every 1 ms from multiple sensors → the interrupt rate is high and periodic; the ISR must be extremely short, and the dispatch mechanism must identify the source instantly, not scan every device.
- Vectored interrupt handling → each sensor is wired to a unique interrupt line/vector, and the interrupt controller automatically jumps to the correct ISR address — removing the need for software polling of a status register to find which device interrupted.
- Memory-mapped vs I/O-mapped → memory-mapped I/O lets the sensor registers be accessed with ordinary load/store instructions (single-cycle, pipelined, usable by any instruction, DMA-compatible), whereas I/O-mapped I/O needs dedicated IN/OUT instructions on a separate, smaller address space and cannot be used by DMA engines directly. For lowest latency, memory-mapped I/O is preferred.

Solution Development

Design a vectored, priority-based interrupt architecture built on memory-mapped I/O:

- Map every sensor's data/status/control register into the CPU's normal memory address space (memory-mapped I/O).
- Connect each sensor to a dedicated line on a vectored interrupt controller (e.g., an NVIC-style controller) so each sensor has its own vector/ISR entry point — no polling loop is needed to identify the source.
- Keep each ISR minimal: read the data register, clear the interrupt flag, push the sample into a buffer, and return — heavier processing is deferred to a low-priority background (bottom-half) task.
- Assign interrupt priorities so that time-critical sensors preempt less critical ones, preventing pile-up under the 1 ms arrival rate.

Rough load check: if each ISR takes ~10 μ s and a sensor fires once every 1000 μ s, that sensor consumes 1% of CPU time; even with 10 such sensors the load is about 10%, comfortably meeting the <15% budget.

Application of Concepts

Vectored interrupt tables, ISR/bottom-half (deferred processing) split, memory-mapped register access, and interrupt-priority scheduling are combined to satisfy the real-time constraint set.

Justification

Memory-mapped I/O avoids the overhead of special I/O instructions and integrates naturally with vectored, low-latency dispatch, while short ISRs and deferred processing keep CPU utilization well under the 15% ceiling — an I/O-mapped, polled design would not meet either the latency or the utilization constraint.

Question 2: High-Speed NVMe/PCIe Storage I/O Subsystem

Problem Understanding

A storage subsystem must sustain throughput above 3 GB/s using NVMe over PCIe, while ensuring the CPU never performs the bulk data copy itself, and interrupt notification is combined with DMA for completion signaling.

Constraint Analysis

- Throughput > 3 GB/s → the PCIe link must supply enough raw bandwidth; a PCIe Gen3 x4 link (~3.94 GB/s) or Gen4 x2/x4 link comfortably exceeds this.
- Must use NVMe over PCIe → NVMe's multiple parallel submission/completion queues (as opposed to AHCI/SATA's single queue) are required to sustain high IOPS and throughput.
- CPU should not handle bulk transfer → the actual data movement must be done by a bus-mastering DMA engine inside the NVMe controller, not by CPU-driven copy loops.
- DMA + interrupt notification → the CPU should only be interrupted once a transfer/block completes, not for every data unit.

Solution Development

Architecture: CPU/Root Complex → PCIe Gen4 x4 link → NVMe SSD controller.

- The CPU writes a command descriptor (source/destination address, length) into an NVMe submission queue in host RAM — this is the only CPU involvement before transfer.
- The NVMe controller's DMA engine performs scatter-gather bus-mastering transfers directly between the SSD and system RAM, bypassing the CPU entirely for the bulk data.
- On completion, the controller writes a completion queue entry and raises an MSI-X interrupt; the CPU's ISR simply reads the small completion entry and signals the requesting application — it never touches the transferred payload.

Application of Concepts

PCIe lane bandwidth calculation, NVMe submission/completion queue model, bus-mastering DMA with scatter-gather, and MSI-X interrupt signaling are applied together.

Justification

A PCIe x4 link provides the required throughput headroom; NVMe's queue-based, DMA-driven model offloads all bulk transfer from the CPU; and MSI-X-based completion interrupts keep CPU involvement to a single lightweight interrupt per transfer, satisfying every constraint simultaneously.

Question 3: Multi-Device USB/Thunderbolt Communication Architecture

Problem Understanding

The architecture must connect 6 USB peripherals and 2 Thunderbolt high-speed devices, avoid starving any device of interrupt service, use both hardware and software interrupts appropriately, and guarantee fair arbitration on the shared bus.

Constraint Analysis

- 6 USB peripherals → USB is inherently host-pollled/scheduled (frames in USB2, microframes in USB3), so bus access for these devices is already time-division managed by the host controller.
- 2 Thunderbolt devices → Thunderbolt tunnels PCIe (and DisplayPort) traffic and supports daisy-chaining, so it needs credit-based, bandwidth-guaranteed arbitration rather than simple polling.
- Avoid interrupt starvation → no single device (especially a high-bandwidth Thunderbolt device) should be able to monopolize the interrupt/bus resources and block the USB peripherals indefinitely.
- Hardware and software interrupts → real device events (data ready, error) should raise hardware interrupts; scheduling/deferred-processing tasks should be signaled by software interrupts running at lower priority.
- Fair bus arbitration → the shared interconnect needs an arbitration policy that guarantees every device a bounded wait time.

Solution Development

- USB side: the host controller uses frame-based scheduling (1 ms frames / 125 μs microframes) with round-robin allocation of transaction slots among the 6 peripherals, which inherently prevents starvation.

- Thunderbolt side: use PCIe-style credit-based flow control with weighted round-robin arbitration across the two high-speed devices, reserving a guaranteed bandwidth share so a bulk Thunderbolt transfer cannot block the bus indefinitely.
- Interrupt layer: hardware interrupts from the USB host controller and Thunderbolt controller are assigned distinct priority levels; software interrupts (soft-IRQs/deferred procedure calls) handle bottom-half processing (e.g., buffer copy-out, protocol stack work) at lower priority so they never block a pending hardware event.
- Global arbiter: implement a rotating-priority (round-robin among equal-priority devices, with aging) arbitration scheme at the root controller so that no device's requests can be perpetually deferred, bounding worst-case wait time for all 8 devices.

Application of Concepts

USB frame-based scheduling, PCIe/Thunderbolt credit-based arbitration, priority-separated hardware/software interrupt handling, and round-robin-with-aging fairness policies are applied.

Justification

Combining USB's built-in frame scheduling with weighted round-robin arbitration for Thunderbolt, and separating time-critical hardware interrupts from deferred software interrupts, guarantees every one of the 8 devices a bounded service time — satisfying both the starvation-avoidance and fairness requirements without sacrificing Thunderbolt's high bandwidth.

Question 4: High-Throughput Data Acquisition System

Problem Understanding

A data acquisition system must sustain a continuous 500 MB/s stream with minimal CPU intervention, requires a reasoned comparison between DMA transfer and interrupt-driven I/O, and must use memory-mapped I/O for the device's control/status registers.

Constraint Analysis

- 500 MB/s continuous stream → an extremely high data rate that must not depend on per-unit CPU involvement.
- Minimal CPU intervention → the transfer mechanism itself should require the CPU only for setup and periodic notification, not for moving the data.
- DMA vs interrupt-driven I/O comparison is explicitly required.
- Memory-mapped I/O for device registers → configuration/control access should use normal load/store instructions.

Solution Development — Quantitative Comparison

Interrupt-driven I/O: if the CPU is interrupted for every data unit (e.g., a 4-byte word) at 500 MB/s, that implies roughly 125 million interrupts per second. Even at an optimistic $\sim 1\text{--}2\ \mu\text{s}$ overhead per interrupt, this alone would require 125–250 seconds of CPU time per second — far more than 100% of one core is available. Pure interrupt-driven I/O is therefore infeasible at this throughput.

DMA transfer: the DMA controller moves data directly between the acquisition device's FIFO and a buffer in RAM in large bursts, and interrupts the CPU only once per completed block. If the block size is 64 KB, the interrupt rate falls to about $500\text{ MB/s} \div 64\text{ KB} \approx 7,600$ interrupts/second — easily serviced with well under 1% CPU load.

Solution Development — Architecture

- Map the acquisition device's control, status, and address/length registers into the memory-mapped I/O address space so the CPU configures transfers with ordinary load/store instructions.
- Use double (ping-pong) buffering: the DMA engine fills buffer A while software drains/processes buffer B, and vice versa, so acquisition never stalls.

- The DMA controller raises one interrupt per buffer-full event; the ISR only swaps buffer pointers and hands the full buffer to a background task.

Application of Concepts

Memory-mapped register access, burst-mode DMA, double/ping-pong buffering, and interrupt-rate/CPU-load estimation are applied to justify the design choice quantitatively.

Justification

The interrupt-count calculation shows interrupt-driven I/O cannot physically keep up with 500 MB/s, whereas block-based DMA reduces CPU interrupts by roughly four orders of magnitude, making DMA the only mechanism able to satisfy the "minimal CPU intervention" constraint at this data rate.

Question 5: Low-Latency Nested Interrupt Handling Mechanism

Problem Understanding

The mechanism must support nested (preemptive) interrupts, guarantee a worst-case interrupt latency below 10 μ s, use vectored interrupts for high-priority devices, and route background work through software interrupts.

Constraint Analysis

- Nested interrupts $\rightarrow$ a higher-priority interrupt must be able to preempt an ISR that is already executing, rather than waiting for it to finish.
- Latency $< 10 \mu$ s $\rightarrow$ every stage between the interrupt signal and the start of ISR execution (vector fetch, context save, priority arbitration) must be tightly bounded.
- Vectored interrupts for high-priority devices $\rightarrow$ these devices need a direct hardware jump to their own ISR, with no software scanning/polling delay.
- Software interrupts for background tasks $\rightarrow$ non-urgent work (scheduling, deferred processing) must run at lower priority so it can never block a high-priority hardware event.

Solution Development

- Use a priority-based nested vectored interrupt controller (NVIC-style): each interrupt source has a priority level; the controller compares an incoming interrupt's priority against the priority of the ISR currently running and preempts automatically if it is higher.
- Only mask interrupts at or below the current priority during an ISR (selective/priority masking), not all interrupts globally — this is what enables true nesting.
- Assign the highest priority levels and dedicated vector entries to time-critical devices so their ISR is reached by a single hardware vector fetch, with no software dispatch loop.
- Bound every critical section (interrupts-disabled region) in the kernel/drivers to a small, known duration so that critical-section time + context-save time + vector-fetch time stays well under the 10 μ s budget.
- Use tail-chaining (jumping directly from one ISR's exit to the next pending ISR's entry without restoring/re-saving context in between) to avoid pipeline-refill overhead on back-to-back interrupts.
- Route background/non-urgent work (e.g., scheduler ticks, deferred bottom-half processing) through software-triggered interrupts (traps/SWI) at the lowest priority, so they are always preemptible by any hardware interrupt.

Application of Concepts

Priority-based preemptive nesting, hardware vector tables, bounded critical sections, tail-chaining, and priority separation between hardware and software interrupts are applied together.

Justification

Hardware vectoring removes software dispatch delay, priority-based (rather than global) interrupt masking is what makes nesting possible, and bounding critical-section length keeps the worst-case path — vector fetch + context save + critical section — under 10 μ s, while low-priority software interrupts guarantee background work never delays a high-priority hardware event.